

Auth & Session Security Analyzer

Rapport d'audit de securite Android

Application analysee : dvba
Type d'authentification : JWT
Vulnerabilites detectees : 37
Date d'analyse : 17/05/2026 00:29

Score global de securite : 0 / 10 (CRITIQUE)

1. Resume Executif

Ce resume presente les principales vulnerabilites identifiees lors de l'audit, evalue le niveau de risque global et formule les recommandations prioritaires.

Checklist OAuth2 générée via Claude

2. Analyse Statique - Vulnerabilites detectees

L'analyse statique consiste en la decompilation de l'APK avec JADX et l'examen des sources Java/Kotlin par patterns regex. Les resultats sont classes par severite : **CRITIQUE** (-3 pts), **ELEVE** (-2 pts), **MOYEN** (-1 pt).

Fichiers analyses : 1165 | Vulnerabilites : 32 | Score : 0/10

Severite	Type	Fichier	Detail
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Parent handler not f...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager n...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager ...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Got null intent from ...
MOYEN	Communication en clair (HTTP)	a.java	http://s
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onCreate: Executor and/or call...
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onActivityResult: Bridge or call...
CRITIQUE	Secret code en dur	b.java	password: "); sb.append(this.f864a.isPassword()); sb.a...
ELEVE	Token dans SharedPreferences	h.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	l.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	t.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	w.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to read GetTokenResponse ...
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to convert GetTokenRespon...
CRITIQUE	Secret code en dur	n1.java	secret="); sb.append(this.i); sb.append("
ELEVE	Identifiants dans les logs	i.java	Log.e(aVar.f1410a, aVar.a("Error parsing token claims", new Obj...
ELEVE	Token dans SharedPreferences	j.java	getSharedPreferences("com.google.firebase.auth
ELEVE	Identifiants dans les logs	k.java	Log.e(aVar2.f1410a, aVar2.a("Unable to decode token", new Obj...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	p.java	getSharedPreferences(String.format("com.google.firebase.auth
ELEVE	Token dans SharedPreferences	AddBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ApproveBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	BankLogin.java	Log.d("accesstoken", string)
ELEVE	Token dans SharedPreferences	BankLogin.java	SharedPreferences sharedPreferences = BankLogin.this.getShar...
ELEVE	Token dans SharedPreferences	BankLogin.java	sharedPreferences.edit().putString("accesstoken
ELEVE	Token dans SharedPreferences	Dashboard.java	SharedPreferences.Editor editorEdit = getApplicationContext().ge...
ELEVE	Token dans SharedPreferences	Myprofile.java	getSharedPreferences("jwt", 0).getString("accesstoken

ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt
ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	SendMoney.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ViewBalance.java	getSharedPreferences("jwt", 0).getString("accesstoken

3. Analyse Dynamique - Tests d'authentification

L'analyse dynamique evalue le comportement de l'application en cours d'exécution. Les tests portent sur la validité des tokens après déconnexion, la durée de vie, la rotation des tokens et la robustesse du mécanisme d'authentification.

Mode d'analyse :	API_DIRECT
Déconnexion effective :	Non (VULNERABLE - rejeu possible)
Durée de vie du token :	None min
Rotation des refresh tokens :	Non testé

Vulnérabilités détectées lors des tests :

CRITIQUE	Le token JWT ne contient pas de champ 'exp'
CRITIQUE	HTTP 200 après logout - invalidation absente
CRITICAL	
PASS	
PASS	

4. Scores MASVS par Domaine

Les scores reflètent le niveau de sécurité par domaine OWASP MASVS v2. Chaque vulnérabilité déduit des points selon sa sévérité. Un score inférieur à 4 indique un risque critique sur ce domaine.

Scores non disponibles (analyse IA désactivée).

5. Checklist de Sécurité (MASVS)

Checklist générée par IA (Claude) adaptée au type d'authentification détecté. Chaque contrôle est mapé à un chapitre OWASP MASVS pertinent. Ces points constituent le référentiel de vérification pour la revue de code.

? Checklist de Sécurité OAuth2 - Android Native Client

Authorization Code Flow + PKCE

> **Date de génération :** 2025 | **Standard de référence :** OWASP MASVS v2.1, OAuth 2.0 RFC 6749/7636

> **Findings statiques intégrés :** 10 findings détectés - ?? 2 CRITIQUE, 6 ÉLEVÉ, 1 MOYEN

? Tableau de Bord des Risques

| Sévérité | Total | Confirmés (statique) | À vérifier (dynamique) |

|-----|-----|-----|-----|

| ? CRITICAL | 3 | 1 | 2 |

| ? HIGH | 6 | 4 | 2 |

| ? MEDIUM | 3 | 1 | 2 |

| ? LOW | 2 | 0 | 2 |

| **TOTAL** | **14** | **6** | **8** |

? SECTION 1 - PKCE (Proof Key for Code Exchange)

...

??

? RFC 7636 - PKCE est OBLIGATOIRE pour les clients publics Android ?

??

...

- **[MASVS-AUTH-1]** Vérifier que le paramètre `code\_challenge` est présent dans chaque requête d'autorisation envoyée au serveur OAuth2 | **Sévérité: CRITICAL** | CWE: 287

- ? *Test :* Intercepter avec Burp Suite le trafic vers `/authorize` et confirmer la présence de `code\_challenge`
- ? *Statut :* ? À vérifier dynamiquement

- **[MASVS-AUTH-1]** Vérifier que `code\_challenge\_method=S256` est utilisé (SHA-256) et NON `plain` - la méthode `plain` n'offre aucune protection réelle | **Sévérité: CRITICAL** | CWE: 326

- ? *Test :* Confirmer dans la requête `/authorize` : `code\_challenge\_method=S256`
- ? *Statut :* ? À vérifier dynamiquement
- ?? *Code review :* Chercher `CodeVerifier`, `S256`, `PLAIN` dans les sources

- **[MASVS-AUTH-1]** Vérifier que le `code\_verifier` est généré de façon cryptographiquement aléatoire (min 43 caractères, max 128, base64url) via `SecureRandom` | **Sévérité: HIGH** | CWE: 338

- ? *Test :* Audit du code source - `new SecureRandom()` obligatoire, jamais `new Random()`
- ? *Statut :* ? À vérifier statiquement

- **[MASVS-AUTH-1]** Vérifier que le `code\_verifier` est stocké de façon sécurisée en mémoire uniquement (jamais persisté sur disque ou dans les logs) entre la requête d'autorisation et l'échange de token | **Sévérité: HIGH** | CWE: 312

- ? *Test :* Vérifier l'absence du verifier dans SharedPreferences, fichiers, ou logs
- ? *Statut :* ? À vérifier

?? SECTION 2 - State Parameter (Protection CSRF)

...

??

? Le paramètre STATE protège contre les attaques CSRF sur le flow OAuth ?

??

...

- **[MASVS-AUTH-2]** Vérifier que le paramètre `state` est présent et généré de façon cryptographiquement aléatoire (min 128 bits via `SecureRandom`) dans chaque requête `/authorize` | **Sévérité: CRITICAL** | CWE: 352

- ? *Test :* Intercepter `/authorize` et confirmer la présence et l'entropie suffisante du `state`
- ? *Statut :* ? À vérifier dynamiquement

- **[MASVS-AUTH-2]** Vérifier que la valeur du `state` reçue dans le callback est comparée à la valeur originale envoyée - toute divergence doit annuler le flow | **Sévérité: CRITICAL** | CWE: 352

- ? *Test :* Modifier manuellement le `state` dans le callback et vérifier que l'application rejette la réponse

- ? *Statut :* ? À vérifier dynamiquement

- **[MASVS-AUTH-2]** Vérifier que le `state` est à usage unique et invalidé après vérification (protection contre le replay) | **Sévérité: HIGH** | CWE: 294

- ? *Test :* Réutiliser le même callback avec le même `state` - doit être rejeté

- ? *Statut :* ? À vérifier

? SECTION 3 - Redirect URI

...

????????????????????????????????

6. Criteres d'Acceptation de Securite

Les criteres d'acceptation definissent les conditions que le systeme doit satisfaire pour etre considere comme securise. Ils servent de base aux tests d'acceptation, aux revues de code et aux audits de conformite MASVS.

Criteres non disponibles.